Cloud DevSecOps Engineer

A to Z Master Notes

A complete, interview-ready reference covering Linux, Networking, Git, Python, AWS, Terraform/OpenTofu, Ansible, Docker, Kubernetes, CI/CD, Application & Pipeline Security, Observability, SRE, and a full End-to-End DevSecOps Project

"Security built into every stage of the pipeline – not bolted on at the end."

Prepared as structured study notes – 52 Modules

September 14, 2026

Contents

1 Foundations: DevSecOps, Networking **6**
- 1.1 Module 1: IT & DevSecOps Fundamentals . . . 6
  - 1.1.1 Core Concepts . . . 6
  - 1.1.2 DevSecOps Lifecycle . . . 7
  - 1.1.3 DevSecOps Engineer – Roles & Responsibilities . . . 7
  - 1.1.4 Environments . . . 7
- 1.2 Module 2: Networking Fundamentals . . . 8
  - 1.2.1 Models . . . 8
  - 1.2.2 Addressing . . . 8
  - 1.2.3 Core Services . . . 9
  - 1.2.4 Transport & Protocols . . . 9
  - 1.2.5 Traffic Management & Security Devices . . . 9
  - 1.2.6 Diagnostic Tools . . . 10
  - 1.2.7 Important Ports . . . 10

2 Linux, Shell Scripting & Git **11**
- 2.1 Module 3: Linux Fundamentals . . . 11
  - 2.1.1 Architecture & Distributions . . . 11
  - 2.1.2 Filesystem Hierarchy . . . 11
  - 2.1.3 Essential Commands . . . 12
- 2.2 Module 4: Linux Administration . . . 12
  - 2.2.1 Core Areas . . . 12
  - 2.2.2 Important Commands . . . 13
- 2.3 Module 5: Shell Scripting . . . 13
  - 2.3.1 Bash Basics . . . 13
  - 2.3.2 Example: Service Health Check . . . 14
- 2.4 Module 6: Git & GitHub . . . 14
  - 2.4.1 Core Theory . . . 14
  - 2.4.2 Common Commands . . . 15
  - 2.4.3 Key Concepts . . . 15

3 Automation with Python & Cloud Fundamentals **17**
- 3.1 Module 7: Python for DevOps . . . 17
  - 3.1.1 Core Language Topics . . . 17
  - 3.1.2 Practical Automation Projects . . . 18

4 Cloud Computing & AWS **19**
- 4.1 Module 8: Cloud Computing Fundamentals . . . 19
  - 4.1.1 Core Concepts . . . 19
  - 4.1.2 Reliability Concepts . . . 20
- 4.2 Module 9: AWS Core . . . 20

4.2.1 IAM – Identity & Access Management . . . 20
4.2.2 EC2 – Elastic Compute Cloud . . . 21
4.2.3 Networking (VPC) . . . 21
4.2.4 Storage . . . 22
4.2.5 Databases . . . 22
4.2.6 Load Balancing & DNS . . . 23
4.3 Module 10: AWS DevSecOps Services . . . 23
4.3.1 Monitoring & Auditing . . . 23
4.3.2 Secrets & Encryption . . . 24
4.3.3 Perimeter, Governance & Compute Platforms . . . 24
4.3.4 Typical Secure Delivery Architecture . . . 24

5 Infrastructure as Code & Configuration Management 25
5.1 Module 11: Terraform + OpenTofu . . . 25
5.1.1 Core Concepts . . . 25
5.1.2 Commands . . . 26
5.1.3 Practical: What to Build . . . 27
5.2 Module 12: Ansible . . . 27
5.2.1 Architecture & Concepts . . . 27
5.2.2 Commands . . . 28
5.2.3 Practical Projects . . . 28

6 Containers: Docker & Kubernetes 29
6.1 Module 13: Docker Fundamentals . . . 29
6.1.1 Core Concepts . . . 29
6.1.2 Commands . . . 29
6.2 Module 14: Docker Advanced . . . 30
6.2.1 Dockerfile Instructions . . . 30
6.2.2 Advanced Topics . . . 30
6.2.3 Image Scanning Tools . . . 31
6.3 Module 15: Kubernetes Fundamentals . . . 31
6.3.1 Architecture . . . 31
6.3.2 Core Objects . . . 31
6.3.3 Commands . . . 32
6.4 Module 16: Kubernetes Advanced . . . 32
6.4.1 Configuration & Storage . . . 32
6.4.2 Workload Types . . . 32
6.4.3 Networking, Scaling & Scheduling . . . 33
6.5 Module 17: Kubernetes Security . . . 33
6.5.1 Access Control . . . 33
6.5.2 Workload & Network Security . . . 34
6.5.3 Security Tools . . . 34
6.6 Module 18: Helm . . . 35
6.6.1 Core Concepts . . . 35
6.6.2 Commands . . . 35

7 CI/CD 36
7.1 Module 19: Jenkins . . . 36
7.1.1 Architecture & Concepts . . . 36
7.1.2 Typical Secure Pipeline . . . 37
7.2 Module 20: GitHub Actions / GitLab CI . . . 37
7.2.1 GitHub Actions . . . 37
7.2.2 GitLab CI/CD . . . 37

8 Core DevSecOps: Pipeline Security & Testing **39**
8.1 Module 21: DevSecOps Pipeline Security . . . 39
8.1.1 Full Secure Pipeline . . . 39
8.2 Module 22: SAST – Static Application Security Testing . . . 39
8.2.1 Tools . . . 39
8.2.2 Key Terms . . . 39
8.3 Module 23: SCA – Software Composition Analysis . . . 40
8.3.1 Key Terms . . . 40
8.3.2 Tools . . . 40
8.4 Module 24: DAST – Dynamic Application Security Testing . . . 40
8.4.1 Tools . . . 41
8.4.2 What DAST Finds . . . 41

9 Secrets & Supply Chain Security **42**
9.1 Module 25: Secrets Security . . . 42
9.1.1 Secret Management Platforms . . . 42
9.1.2 Secret Scanning Tools . . . 42
9.2 Module 26: Software Supply Chain Security . . . 43
9.2.1 Key Concepts . . . 43
9.2.2 Tools & Standards . . . 43

10 Cloud Security & Policy as Code **44**
10.1 Module 27: Cloud Security . . . 44
10.1.1 Core Controls . . . 44
10.1.2 Guiding Principles . . . 45
10.2 Module 28: Policy as Code . . . 45
10.2.1 Frameworks . . . 45
10.2.2 Security Scanning Tools for IaC . . . 45
10.2.3 Example Policies . . . 46

11 Observability: Monitoring, Logging & Tracing **47**
11.1 Module 29: Monitoring . . . 47
11.1.1 Core Concepts . . . 47
11.1.2 Tools . . . 47
11.2 Module 30: Centralized Logging . . . 48
11.2.1 Stack Components . . . 48
11.3 Module 31: OpenTelemetry . . . 48
11.3.1 Core Concepts . . . 48
11.4 Module 32: SIEM & Runtime Security . . . 49
11.4.1 Core Concepts . . . 49
11.4.2 Tools . . . 49
11.4.3 Example Detections . . . 49

12 Compliance & Reliability **50**
12.1 Module 33: Governance, Risk & Compliance (GRC) . . . 50
12.1.1 Key Frameworks (at least basics) . . . 50
12.1.2 Core GRC Terms . . . 50
12.2 Module 34: SRE Concepts . . . 51
12.2.1 Core Terms . . . 51

13 Deployment Strategies & GitOps **53**
13.1 Module 35: Deployment Strategies . . . 53
13.2 Module 36: GitOps . . . 53
13.2.1 Tools . . . 53

13.2.2 Key Concepts . . . 54

14 Managed Kubernetes, Serverless & Data 55
14.1 Module 37: AWS EKS . . . 55
14.1.1 Detailed Concepts . . . 55
14.2 Module 38: Serverless DevSecOps . . . 56
14.2.1 AWS Serverless Building Blocks . . . 56
14.2.2 Serverless Security . . . 56
14.3 Module 39: Database for DevSecOps . . . 56
14.3.1 Engine Types . . . 56
14.3.2 Operational Concepts . . . 57
14.4 Module 40: Application Architecture . . . 57
14.4.1 Architectural Patterns . . . 57

15 Production Troubleshooting & Incident Handling 59
15.1 Module 41: Production Troubleshooting . . . 59
15.1.1 General Infrastructure Scenarios . . . 59
15.1.2 Web & DNS/SSL Scenarios . . . 59
15.1.3 Container & Kubernetes Scenarios . . . 60
15.1.4 Pipeline & Cloud Scenarios . . . 60
15.2 Module 42: Security Incident Handling . . . 61
15.2.1 Example Incidents & Response Focus . . . 61

16 High Availability, DR & Cost Optimization 62
16.1 Module 43: High Availability Architecture . . . 62
16.1.1 Core Concepts Recap . . . 62
16.2 Module 44: Backup & Disaster Recovery . . . 62
16.2.1 Core Terms . . . 62
16.2.2 DR Strategies (increasing cost, decreasing RTO) . . . 63
16.3 Module 45: FinOps Basics . . . 63
16.3.1 Key Tools & Levers . . . 63

17 Certificates, Identity & Kubernetes Networking 65
17.1 Module 46: SSL/TLS . . . 65
17.1.1 Core Concepts . . . 65
17.1.2 Tools & Services . . . 65
17.2 Module 47: Authentication & Authorization . . . 66
17.2.1 Core Concepts . . . 66
17.3 Module 48: Kubernetes Networking . . . 66
17.3.1 Traffic Patterns . . . 67
17.3.2 Service Types & Components . . . 67
17.3.3 Common CNIs . . . 67
17.4 Module 49: Service Mesh . . . 67
17.4.1 Core Concepts . . . 67

18 Core Principles, Capstone Project & Interview Prep 69
18.1 Module 50: Core Security Concepts . . . 69
18.1.1 Must-Remember Principles . . . 69
18.2 Module 51: End-to-End DevSecOps Project . . . 70
18.2.1 Application Delivery Pipeline . . . 70
18.2.2 Infrastructure Layer . . . 70
18.2.3 Security Layer Woven Throughout . . . 70
18.3 Module 52: Interview Notes . . . 70
18.3.1 1. Concept Questions . . . 70

18.3.2 2. Scenario Questions 71
18.3.3 3. Commands Cheat-Sheet 71
18.3.4 4. Real-Time Architecture Questions 72

Chapter 1

Foundations: DevSecOps, Networking

1.1 Module 1: IT & DevSecOps Fundamentals

1.1.1 Core Concepts

IT Infrastructure
The complete set of hardware, software, networks, data centers and facilities used to develop, test, deliver, and support IT services – servers, storage, networking equipment, and the OS/middleware running on them.

Development vs Operations
Development (Dev) builds and writes the application code; Operations (Ops) deploys, runs, and maintains that code reliably in production. Historically these were separate teams with conflicting goals (speed vs stability).

DevOps
A culture and set of practices that unifies Dev and Ops using automation, CI/CD, and shared ownership so that software can be built, tested, and released faster and more reliably.

DevSecOps
DevOps extended so that security is embedded into every stage of the software delivery lifecycle (code, build, test, deploy, run) instead of being a final gate-check before release.

DevOps vs DevSecOps
DevOps optimizes for speed and collaboration between Dev and Ops. DevSecOps adds a third pillar – Security – so that vulnerabilities are caught early ("shift left") and monitored continuously in production.

SDLC
Software Development Life Cycle – the phases a piece of software passes through: Requirements, Design, Development, Testing, Deployment, Maintenance.

Agile
An iterative approach to software delivery that breaks work into small increments, gets frequent feedback, and adapts plans continuously instead of following one rigid upfront plan (as in Waterfall).

Scrum
A popular Agile framework organizing work into fixed-length iterations (Sprints) with defined roles (Product Owner, Scrum Master, Development Team) and ceremonies (Sprint Planning, Daily Standup, Sprint Review, Retrospective).

Sprint
A fixed, short time-box (commonly 1–4 weeks) in Scrum during which a specific set of backlog items is designed, built, and tested.

CI/CD
Continuous Integration (frequently merging and automatically building/testing code) plus Continuous Delivery/Deployment (automatically packaging and releasing that code to staging/production).

Shift Left Security
Moving security checks (code scanning, dependency scanning, secret scanning) as early as possible in the pipeline – ideally at commit/build time – rather than only just before release.

Shift Right Security
Extending security practices into production – runtime monitoring, threat detection, chaos/security testing on live systems – to catch what static, pre-release checks miss.

1.1.2 DevSecOps Lifecycle

```
Plan -> Code -> Build -> Test -> Release -> Deploy -> Operate -> Monitor
(Security checks are injected at every one of these stages:  threat modeling
in Plan, SAST/secret-scan in Code, SCA in Build, DAST in Test, image
signing in Release, policy-as-code in Deploy, RBAC/network-policy in
Operate, SIEM/runtime security in Monitor.)
```

1.1.3 DevSecOps Engineer – Roles & Responsibilities

- Design and maintain CI/CD pipelines with integrated security gates (SAST, SCA, secret scanning, container scanning, DAST).
- Provision and manage cloud infrastructure using Infrastructure as Code (Terraform/OpenTofu, CloudFormation).
- Harden Kubernetes clusters – RBAC, network policies, pod security standards, admission control.
- Manage secrets securely (Vault, AWS Secrets Manager) and enforce least-privilege IAM.
- Set up monitoring, logging, and alerting (Prometheus/Grafana, CloudWatch, SIEM).
- Respond to security incidents, patch vulnerabilities, and drive compliance (SOC2, ISO 27001, CIS Benchmarks).
- Automate repetitive operational tasks using shell/Python scripting and configuration management (Ansible).

1.1.4 Environments

Production Environment
The live environment that real end-users interact with. Requires maximum stability, monitoring, and controlled change (via CI/CD, not manual edits).

Dev
Where developers write and unit-test code; frequently changing, least stable.

QA
Where testers validate functionality against requirements, often with automated test suites.

UAT (User Acceptance Testing)
Business/end-users validate that the system meets their real-world needs before go-live.

Staging
A near-identical replica of production used for final integration/performance testing before release.

Production
The customer-facing, revenue-impacting environment – changes here follow strict change-management and rollback plans.

1.2 Module 2: Networking Fundamentals

1.2.1 Models

OSI Model
A 7-layer conceptual model describing how data moves across a network: 7 Application, 6 Presentation, 5 Session, 4 Transport, 3 Network, 2 Data Link, 1 Physical. Used to reason about where a networking problem lives.

TCP/IP Model
The practical 4-layer model actually used on the Internet: Application, Transport, Internet, Network Access. Maps loosely onto OSI but merges several layers.

1.2.2 Addressing

IP Address
A unique numeric identifier assigned to a device on a network so it can send/receive traffic.

IPv4 / IPv6
IPv4 uses 32-bit addresses (e.g. 192.168.1.10, 4.3 billion addresses, now scarce). IPv6 uses 128-bit addresses (e.g. 2001:db8::1) to solve address exhaustion and simplify routing.

Public IP / Private IP
A public IP is globally routable on the Internet. A private IP (10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16) is only valid inside a private network and needs NAT to reach the Internet.

Static / Dynamic IP
A static IP never changes (used for servers). A dynamic IP is assigned automatically (often via DHCP) and can change over time.

Subnet
A logically segmented portion of a larger network, used to organize hosts and control traffic/broadcast domains.

CIDR
Classless Inter-Domain Routing – notation like 10.0.0.0/24 that specifies a network address plus the number of bits used for the network portion.

Subnet Mask
A 32-bit value (e.g. 255.255.255.0) that separates the network portion of an IP address from the host portion.

Gateway
The router/device a host sends traffic to when the destination is outside its own subnet.

1.2.3 Core Services

DNS
Domain Name System – translates human-readable domain names (example.com) into IP addresses.

DHCP
Dynamic Host Configuration Protocol – automatically assigns IP addresses, subnet masks, gateways, and DNS servers to devices joining a network.

NAT
Network Address Translation – rewrites private IP addresses to a public IP (and back) so internal hosts can reach the Internet without each needing a public IP.

Route Tables
A set of rules on a router/instance that decide which path (next hop) traffic takes based on the destination IP.

1.2.4 Transport & Protocols

TCP vs UDP
TCP is connection-oriented, reliable, ordered (handshake + retransmission) – used for HTTP, SSH, databases. UDP is connectionless, faster, no delivery guarantee – used for DNS lookups, video/voice streaming.

Ports
A number (0–65535) identifying a specific process/service on a host so multiple services can share one IP.

HTTP / HTTPS
HTTP is the plaintext protocol for web traffic. HTTPS is HTTP secured with TLS/SSL encryption.

SSL / TLS
Cryptographic protocols that encrypt data in transit and authenticate the server (and optionally client) using certificates. TLS is the modern, secure successor to SSL.

SSH
Secure Shell – an encrypted protocol for remote command-line access and secure file transfer/-tunneling.

FTP / SFTP
FTP is an old, unencrypted file-transfer protocol. SFTP transfers files over an encrypted SSH connection.

1.2.5 Traffic Management & Security Devices

Load Balancer
Distributes incoming traffic across multiple backend servers to improve availability, scalability, and fault tolerance.

Reverse Proxy
Sits in front of backend servers and forwards client requests to them, often adding caching, SSL termination, or load balancing (e.g. Nginx).

Proxy Server
Sits in front of clients (forward proxy) and forwards their requests outward – used for filtering, caching, or anonymizing client traffic.

Firewall
Filters inbound/outbound traffic based on rules (IP, port, protocol) to block unauthorized access.

VPN
Virtual Private Network – creates an encrypted tunnel over a public network so remote users/sites can securely reach a private network.

Bastion Host
A hardened, tightly-controlled server exposed to the Internet that acts as the single entry point (jump box) into a private network – limits the attack surface.

1.2.6 Diagnostic Tools

ICMP
Internet Control Message Protocol – used for diagnostic/error messages, most famously by `ping`.

ping
Tests reachability of a host and measures round-trip latency.

traceroute
Shows the path (hop by hop) packets take to reach a destination – useful to find where connectivity breaks.

nslookup / dig
Query DNS servers to resolve a domain name (dig gives more detailed output, preferred on Linux).

curl
Command-line tool to send HTTP(S)/FTP requests and inspect responses – essential for testing APIs and web endpoints.

telnet / nc
Used to manually test whether a TCP port on a remote host is open and reachable.

1.2.7 Important Ports

Port	Service
22	SSH
80	HTTP
443	HTTPS
53	DNS
3306	MySQL
5432	PostgreSQL
6379	Redis
8080	Application (common alt-HTTP)
6443	Kubernetes API Server

Chapter 2

Linux, Shell Scripting & Git

2.1 Module 3: Linux Fundamentals

2.1.1 Architecture & Distributions

Linux Architecture
Layered as: Hardware → Kernel (manages processes, memory, devices, filesystem) → Shell (interprets user commands) → Applications/Utilities.

Ubuntu
A Debian-based distribution, very common for servers and DevOps tooling; uses `apt` for packages.

RHEL
Red Hat Enterprise Linux – an enterprise-grade distribution popular in corporate/regulated environments; uses `yum`/`dnf`.

Amazon Linux
AWS's own RHEL/Fedora-derived distribution, optimized for and tightly integrated with EC2.

2.1.2 Filesystem Hierarchy

/etc
Configuration files for the system and installed applications.

/var
Variable data that changes frequently – logs (`/var/log`), spool files, caches.

/home
Personal directories for each user.

/opt
Optional/third-party software packages installed outside the standard package manager.

/tmp
Temporary files, usually cleared on reboot.

/proc
A virtual filesystem exposing kernel and process information in real time.

/usr
User-space programs, libraries, and documentation shared system-wide.

2.1.3 Essential Commands

```
pwd # print current working directory
ls # list directory contents
cd # change directory
mkdir # create a directory
rm # remove files/directories
cp # copy files/directories
mv # move or rename files
cat # print file content
less # view file content page by page
head # show first N lines of a file
tail # show last N lines of a file (tail -f to follow live)
touch # create an empty file / update timestamp
find # search for files by name/type/size/time
grep # search text using patterns (regex)
awk # pattern scanning + text processing language (columns)
sed # stream editor for find/replace on text
sort # sort lines of text
uniq # remove/report duplicate adjacent lines
wc # count lines, words, bytes (wc -l for line count)
cut # extract columns/fields from text
```

2.2 Module 4: Linux Administration

2.2.1 Core Areas

Users & Groups
Every process runs as a user; users belong to groups. Access to files/resources is controlled by user/group ownership plus permissions.

File Permissions
Each file has read (r), write (w), execute (x) permissions for Owner, Group, and Others – shown as e.g. `rwxr-xr-`.

chmod
Changes file permissions, e.g. `chmod 755 file` (owner: rwx, group/others: r-x).

chown
Changes file ownership, e.g. `chown user:group file`.

sudo
Allows a permitted user to run a command with another user's (usually root) privileges, with an audit trail.

Processes
Running instances of programs, each with a PID; can be foreground/background, and controlled with signals (e.g. `kill`).

Services & systemd
`systemd` is the modern init system managing services ("units") – starting, stopping, enabling on boot, and tracking their status.

Packages
Software is installed/updated/removed via package managers: `apt` (Debian/Ubuntu), `yum`/`dnf` (RHEL/Amazon Linux).

Logs
System and application logs live under `/var/log` or are captured by `systemd`'s journal, viewed with `journalctl`.

Disks, Memory, CPU
Core resources to monitor for performance/capacity issues; tracked with `df`, `free`, `top`, etc.

2.2.2 Important Commands

```
ps # list running processes
top / htop# live view of CPU/memory usage per process
free # show memory usage (RAM/swap)
df # show disk space usage per filesystem
du # show disk usage of files/directories
lsblk # list block devices (disks/partitions)
mount # attach a filesystem to a directory
systemctl # start/stop/enable/status of systemd services
journalctl# view systemd logs
netstat # legacy tool to show network connections/ports
ss # modern replacement for netstat
lsof # list open files (incl. by which process/port)
ip # show/configure network interfaces, routes, addresses
curl # make HTTP(S) requests from the CLI
wget # download files over HTTP(S)/FTP

# Examples
chmod 755 file
chown user:group file
systemctl status nginx
journalctl -u nginx
```

2.3 Module 5: Shell Scripting

2.3.1 Bash Basics

Variables
Defined as `NAME=value` (no spaces), referenced as `$NAME`.

Environment Variables
Variables exported to child processes with `export NAME=value`; visible system-wide within that shell's process tree (e.g. `PATH`, `HOME`).

Arguments
Positional parameters passed to a script: `$1`, `$2`...; `$0` is the script name; `$@`/`$*` are all arguments; `$#` is the argument count.

Conditions
`if`/`elif`/`else`/`fi` blocks test exit codes or expressions (`[ ... ]` or `[[ ... ]]`).

Loops
`for`, `while`, and `until` loops iterate over lists, ranges, or conditions.

Functions
Reusable blocks defined with `function_name() { ... }`, called by name.

Exit Codes
Every command returns a numeric exit status (`$?`); `0` means success, non-zero means failure – used for error handling and chaining (`&&`, `||`).

Error Handling
`set -e` (exit on error), `set -u` (error on unset variable), `trap` (run cleanup code on exit/signal).

grep/awk/sed
Used inside scripts for filtering, extracting, and transforming text output.

cron jobs
Scheduled, recurring tasks configured via crontab (`minute hour day month weekday command`).

Logging
Scripts should write timestamped output to a log file (or `syslog`/`journal`) for later troubleshooting.

2.3.2 Example: Service Health Check

```
#!/bin/bash

if systemctl is-active nginx
then
    echo "Nginx Running"
else
    systemctl start nginx
fi
```

Note

Practical scripts a DevSecOps engineer should be able to write from scratch: disk-usage alert, CPU monitoring, backup automation, log cleanup, service health check, deployment script, user creation, and general server monitoring. Each should log its actions and exit with a meaningful exit code so it can be chained into cron or a pipeline.

2.4 Module 6: Git & GitHub

2.4.1 Core Theory

Version Control
A system that tracks changes to files over time, allowing you to recall specific versions, collaborate, and merge concurrent changes.

Git Architecture
Distributed VCS: every clone is a full copy of the repository history (unlike centralized systems like SVN).

Working Directory
The actual files on disk that you edit.

Staging Area (Index)
A holding area where changes are prepared (`git add`) before being committed.

Local Repository
The `.git` folder holding the full committed history on your machine.

Remote Repository
A shared copy of the repository (e.g. on GitHub/GitLab) used for collaboration.

2.4.2 Common Commands

```
git init # create a new local repository
git clone # copy a remote repository locally
git status # show changed/staged/untracked files
git add # stage changes for commit
git commit # save staged changes to local history
git push # upload local commits to the remote
git pull # fetch + merge remote changes into current branch
git fetch # download remote changes without merging
git merge # combine another branch's history into the current one
git rebase # replay commits on top of another branch (linear history)
git log # view commit history
git diff # show unstaged/uncommitted changes
git stash # temporarily shelve uncommitted changes
git reset # move HEAD/undo commits (can discard changes)
git revert # create a new commit that undoes a previous commit safely
git cherry-pick # apply a specific commit from another branch
```

2.4.3 Key Concepts

Branching
Creating an isolated line of development off the main history, e.g. for a feature or bugfix.

Merge vs Rebase
Merge preserves both histories and creates a merge commit; rebase rewrites commits onto a new base for a cleaner, linear history (but rewrites SHAs – avoid on shared branches).

Merge Conflicts
Occur when the same lines are changed differently on two branches being merged; must be resolved manually.

Pull Requests (PRs)
A request to merge one branch into another, used as the checkpoint for code review, CI checks, and discussion before merging.

Tags
Fixed pointers to a specific commit, typically used to mark releases (e.g. `v1.2.0`).

Releases
A packaged, versioned snapshot of the software (often built on a tag) with release notes and artifacts.

GitFlow
A branching model with long-lived `develop`/`main` branches plus feature/release/hotfix branches – good for scheduled releases.

Trunk Based Development
Everyone commits frequently (often daily) to a single main branch using short-lived feature branches and feature flags – suits continuous delivery.

Branch Protection
Rules on GitHub/GitLab that require PR reviews, passing CI checks, or signed commits before

merging into a protected branch (e.g. `main`).

Code Review
Peer inspection of proposed changes before merge – catches bugs, security issues, and maintains code quality/standards.

GitHub Actions
GitHub's built-in CI/CD automation engine, triggered by repository events (push, PR, schedule).

GitHub Secrets
Encrypted environment variables stored at the repo/org level, injected into workflows without exposing them in code.

```
Developer -> Feature Branch -> Pull Request -> Code Review -> Merge -> Pipeline
```

Chapter 3

Automation with Python & Cloud Fundamentals

3.1 Module 7: Python for DevOps

Note

A DevSecOps engineer does not need full application-developer-level Python, but must be strong at automation-level scripting: talking to APIs, parsing data, and driving cloud SDKs.

3.1.1 Core Language Topics

Variables & Data Types
`int`, `float`, `str`, `bool` – Python is dynamically typed.

List
An ordered, mutable collection: `[1, 2, 3]`.

Dictionary
Key-value mapping: `{"name": "prod"}` – ideal for JSON-like data.

Tuple
An ordered, immutable collection: `(1, 2)`.

Set
An unordered collection of unique elements, good for membership tests and de-duplication.

Conditions & Loops
`if/elif/else`; `for` loops over iterables; `while` loops on a condition.

Functions
Reusable blocks defined with `def`, supporting default args, `*args`/`**kwargs`.

Files
Reading/writing with `open(path, mode)`, preferably using a `with` block to auto-close.

JSON
`json.load`/`json.dumps` to parse/serialize JSON – the native language of most cloud/REST APIs.

YAML
Human-friendly config format (used by Kubernetes, Ansible, CI pipelines) parsed via the `PyYAML`

library.

Exception Handling
`try/except/finally` to catch and gracefully handle runtime errors instead of crashing.

Modules
Reusable `.py` files imported with `import`, allowing code to be organized and shared.

APIs & requests
The `requests` library is the standard way to call REST APIs (GET/POST/PUT/DELETE) from Python.

os & subprocess
`os` interacts with the operating system (env vars, paths); `subprocess` runs external shell commands from Python.

boto3
The official AWS SDK for Python – used to automate EC2, S3, IAM, and virtually every AWS service programmatically.

3.1.2 Practical Automation Projects

- AWS resource automation (start/stop EC2, tag resources) using `boto3`.
- A log parser that extracts errors/patterns from application logs.
- A server-health script that checks CPU/memory/disk and alerts if thresholds are crossed.
- An API monitoring script that pings endpoints and reports downtime.
- An S3 backup automation script that uploads and rotates backups.
- An IAM auditing script that flags overly-permissive policies or unused access keys.

Chapter 4

Cloud Computing & AWS

4.1 Module 8: Cloud Computing Fundamentals

4.1.1 Core Concepts

Cloud Computing
On-demand delivery of compute, storage, networking, and services over the Internet with pay-as-you-go pricing, instead of owning physical hardware.

On-Prem vs Cloud
On-prem means the organization owns and manages its own data-center hardware; cloud shifts infrastructure management to a provider (AWS/Azure/GCP).

IaaS
Infrastructure as a Service – provider gives raw compute/storage/network (e.g. EC2, VPC); you manage the OS upward.

PaaS
Platform as a Service – provider manages the OS/runtime; you just deploy code (e.g. Elastic Beanstalk, Heroku).

SaaS
Software as a Service – fully managed application, ready to use (e.g. Gmail, Salesforce).

Public / Private / Hybrid / Multi Cloud
Public: shared infrastructure from a provider (AWS). Private: dedicated infrastructure for one org. Hybrid: mix of on-prem + cloud. Multi-cloud: using more than one cloud provider simultaneously.

Regions
Independent, geographically separate areas where a cloud provider operates data centers.

Availability Zones (AZs)
Isolated, physically separate data centers within a region, connected with low-latency links – used for high availability.

Edge Locations
Smaller, globally distributed sites (used by CDNs like CloudFront) that cache content closer to end-users.

4.1.2 Reliability Concepts

High Availability
Designing systems to remain operational with minimal downtime, typically via redundancy across AZs.

Scalability
The ability of a system to handle increased load by adding resources (vertically or horizontally).

Elasticity
The ability to automatically scale resources up or down based on real-time demand.

Fault Tolerance
The ability of a system to keep functioning correctly even when a component fails.

Disaster Recovery (DR)
Plans and tooling to restore systems after a catastrophic failure or outage.

RTO
Recovery Time Objective – the maximum acceptable time to restore service after a failure.

RPO
Recovery Point Objective – the maximum acceptable amount of data loss, measured in time (e.g. last 15 minutes of data).

Shared Responsibility Model
AWS secures "the cloud" (hardware, global infrastructure); the customer secures "in the cloud" (data, IAM, OS patching, network configuration, application security).

4.2 Module 9: AWS Core

4.2.1 IAM – Identity & Access Management

Users
Individual identities (human or app) with credentials for accessing AWS.

Groups
Collections of users that share the same permissions.

Roles
An identity with permissions that can be assumed temporarily by users, services, or applications – no long-lived credentials.

Policies
JSON documents that define what actions are allowed/denied on which resources.

Permission Boundaries
An advanced IAM feature that sets the maximum permissions an identity can ever have, regardless of its attached policies.

MFA
Multi-Factor Authentication – requires a second verification factor (e.g. OTP) in addition to a password.

IAM Best Practices
Enforce least privilege, use roles instead of long-lived keys, enable MFA, rotate credentials, avoid

the root account for daily work.

Least Privilege
Grant only the exact permissions required to perform a task – nothing more.

Access Keys
Long-lived programmatic credentials (Access Key ID + Secret) – higher risk if leaked; prefer roles/STS where possible.

AssumeRole & STS
STS (Security Token Service) issues short-lived, temporary credentials when a trusted identity assumes a role – the secure pattern for cross-account/service access.

4.2.2 EC2 – Elastic Compute Cloud

Instances
Virtual servers running in AWS.

AMI
Amazon Machine Image – a template (OS + software) used to launch instances.

Instance Types
Hardware configurations (CPU/RAM/network) optimized for general purpose, compute, memory, or storage workloads.

EBS
Elastic Block Store – persistent block storage volumes attached to EC2 instances.

Security Groups
Virtual, stateful firewalls controlling inbound/outbound traffic at the instance level.

Key Pair
Public/private key used for secure SSH access to an instance.

User Data
A script passed at launch time to automatically bootstrap/configure an instance.

Auto Scaling
Automatically adds/removes EC2 instances based on demand or schedules.

Launch Templates
Reusable, versioned configuration (AMI, instance type, security groups) used by Auto Scaling to launch instances consistently.

4.2.3 Networking (VPC)

VPC
Virtual Private Cloud – an isolated, logically-defined network within AWS where you control IP ranges, subnets, and routing.

Public / Private Subnet
A public subnet has a route to an Internet Gateway (reachable from the Internet); a private subnet does not.

Route Tables
Define how traffic is routed between subnets, gateways, and other networks.

Internet Gateway
Allows resources in public subnets to send/receive traffic to/from the Internet.

NAT Gateway
Allows instances in a private subnet to initiate outbound Internet traffic (e.g. for updates) without being directly reachable from outside.

NACL
Network Access Control List – a stateless firewall operating at the subnet level.

VPC Peering
Directly connects two VPCs so resources can communicate using private IPs.

Transit Gateway
A central hub that simplifies connecting many VPCs and on-prem networks together.

VPC Endpoint / PrivateLink
Allows private connectivity to AWS services (like S3) without traversing the public Internet.

4.2.4 Storage

S3
Simple Storage Service – highly durable object storage for files/objects, accessed via API/HTTP.

EBS
Block storage attached to a single EC2 instance (like a virtual hard disk).

EFS
Elastic File System – a shared, scalable NFS-based file system usable by multiple instances simultaneously.

Storage Classes
Different S3 tiers (Standard, Infrequent Access, Glacier) balancing cost vs retrieval speed.

Versioning
Keeps multiple versions of an S3 object, protecting against accidental overwrite/delete.

Lifecycle Policies
Automatically transition or expire objects between storage classes over time.

Encryption
Protects data at rest (SSE-S3, SSE-KMS) and in transit (TLS).

4.2.5 Databases

RDS
Managed relational database service (MySQL, PostgreSQL, etc.) – AWS handles patching, backups, failover.

Aurora
AWS's own high-performance, MySQL/PostgreSQL-compatible managed database engine.

DynamoDB
Fully managed, serverless NoSQL key-value/document database with single-digit millisecond latency.

ElastiCache
Managed in-memory caching service (Redis/Memcached) for low-latency data access.

4.2.6 Load Balancing & DNS

ALB
Application Load Balancer – Layer 7, routes HTTP(S) traffic based on host/path rules.

NLB
Network Load Balancer – Layer 4, handles very high throughput/low latency TCP/UDP traffic.

Target Groups
A set of registered backend targets (instances, IPs, Lambda) that a load balancer routes to.

Health Checks
Periodic checks the load balancer uses to determine if a target is healthy enough to receive traffic.

Route 53
AWS's managed DNS service.

Hosted Zones
A container for DNS records for a domain within Route 53.

Records
Individual DNS entries (A, CNAME, MX, etc.).

Routing Policies
Rules for how Route 53 answers DNS queries – Simple, Weighted, Latency-based, Failover, Geolocation.

4.3 Module 10: AWS DevSecOps Services

4.3.1 Monitoring & Auditing

CloudWatch
Collects metrics, logs, and events; supports dashboards and alarms.

CloudTrail
Records every API call made in an AWS account – essential for security auditing and forensics.

Config
Continuously records resource configurations and evaluates them against compliance rules.

GuardDuty
Managed threat-detection service that analyzes logs (CloudTrail, VPC Flow Logs, DNS) for malicious activity.

Inspector
Automated vulnerability scanning for EC2 instances and container images.

Security Hub
Aggregates and prioritizes security findings from GuardDuty, Inspector, Config, and other tools in one dashboard.

IAM Access Analyzer
Identifies resources shared with external entities and unused permissions.

4.3.2 Secrets & Encryption

KMS
Key Management Service – create and control encryption keys used across AWS services.

Secrets Manager
Securely stores, rotates, and retrieves secrets like DB credentials and API keys.

Systems Manager (SSM)
A suite of tools for operational management – patching, automation, session access without SSH.

Parameter Store
Part of SSM; stores configuration data and secrets (plain or encrypted) as key-value pairs.

4.3.3 Perimeter, Governance & Compute Platforms

WAF
Web Application Firewall – filters malicious HTTP traffic (SQLi, XSS) in front of applications.

Shield
DDoS protection service for AWS resources.

Organizations
Centrally manage and govern multiple AWS accounts.

Control Tower
Sets up and governs a secure, compliant multi-account AWS environment automatically.

SCP
Service Control Policies – org-level guardrails that set the maximum allowed permissions across accounts.

ECR
Elastic Container Registry – managed Docker image registry.

ECS
Elastic Container Service – AWS-native container orchestration.

EKS
Elastic Kubernetes Service – managed Kubernetes control plane on AWS.

Lambda
Serverless compute – runs code in response to events without managing servers.

CodeBuild / CodeDeploy / CodePipeline
AWS-native CI/CD services for building, deploying, and orchestrating release pipelines.

4.3.4 Typical Secure Delivery Architecture

```
Developer -> GitHub -> CI Pipeline -> Security Scan -> Docker Build
-> ECR -> EKS -> ALB -> Users
```

Chapter 5

Infrastructure as Code & Configuration Management

5.1 Module 11: Terraform + OpenTofu

Note

OpenTofu is an open-source fork of Terraform. Both follow the same declarative Infrastructure-as-Code model: **Write** → **Plan** → **Apply**. Reusable modules are important in both for standardizing how infrastructure is built across teams.

5.1.1 Core Concepts

Infrastructure as Code (IaC)
Managing infrastructure through machine-readable definition files instead of manual, click-based provisioning – enabling version control, review, and repeatability.

Declarative vs Imperative
Declarative (Terraform/OpenTofu) describes the desired end-state and lets the tool figure out how to get there. Imperative (a shell script) describes the exact steps to execute.

HCL
HashiCorp Configuration Language – the syntax used to write Terraform/OpenTofu configuration files.

Providers
Plugins that let Terraform/OpenTofu talk to a specific platform's API (e.g. the AWS provider, Kubernetes provider).

Resources
The individual infrastructure objects being managed (e.g. `aws_instance`, `aws_s3_bucket`).

Variables
Parameterize configuration so the same code can be reused across environments.

Outputs
Expose values (e.g. an instance's IP) from a module/configuration for use elsewhere.

Data Sources
Read-only lookups of existing infrastructure not managed by the current configuration.

Local Values
Named expressions to avoid repeating the same logic multiple times in a configuration.

State
A file that maps real-world resources to your configuration, tracking what Terraform/OpenTofu currently manages.

Remote State & Backend
Storing the state file remotely (e.g. S3) instead of locally, so teams can collaborate safely.

State Locking
Prevents two people from running `apply` at the same time and corrupting the state (e.g. via DynamoDB lock table).

Modules
Reusable, self-contained bundles of Terraform/OpenTofu configuration – the key to standardizing infrastructure.

Workspaces
Allow a single configuration to manage multiple distinct state files (e.g. per environment).

Dependencies
Terraform/OpenTofu automatically builds a dependency graph so resources are created/destroyed in the right order.

count / for_each
Meta-arguments to create multiple instances of a resource/module from a single block.

lifecycle
Block controlling special resource behavior, e.g. `create_before_destroy`, `prevent_destroy`.

import
Brings an existing, manually-created resource under Terraform/OpenTofu management.

taint / replace
Marks a resource to be destroyed and recreated on the next apply.

5.1.2 Commands

```
terraform init # download providers, set up backend
terraform fmt # auto-format configuration files
terraform validate # check configuration syntax/internal consistency
terraform plan # preview changes before applying
terraform apply # create/update infrastructure
terraform destroy # tear down managed infrastructure
terraform output # show output values
terraform state list # list resources tracked in state
terraform import # bring existing infra under management

# OpenTofu (drop-in compatible CLI)
tofu init
tofu plan
tofu apply
tofu destroy
```

5.1.3 Practical: What to Build

- VPC with public/private Subnets, NAT Gateway, Security Groups
- EC2 instances and an Application Load Balancer (ALB)
- Auto Scaling Group and RDS database
- S3 buckets, an EKS cluster, and the IAM roles/policies backing it all

5.2 Module 12: Ansible

5.2.1 Architecture & Concepts

Ansible Architecture
Agentless – the Control Node pushes configuration to Managed Nodes over SSH (no agent needs to be installed on targets).

Control Node
The machine where Ansible is installed and playbooks are run from.

Managed Nodes
The target servers being configured.

Inventory
A file (INI/YAML) listing managed nodes, grouped logically (e.g. `[webservers]`).

Ad-hoc Commands
One-off commands run directly against hosts without writing a playbook.

YAML & Playbooks
Playbooks are YAML files describing a set of automation tasks to run against hosts.

Tasks
Individual units of work inside a playbook (e.g. install a package, copy a file).

Modules
Reusable units of Ansible code that actually perform the work (e.g. `apt`, `copy`, `service`).

Variables & Facts
Variables parameterize playbooks; Facts are auto-discovered system information about a managed node (OS, IP, memory).

Templates
Jinja2-based files (`.j2`) that generate configuration files dynamically using variables.

Handlers
Tasks that only run when notified by another task (e.g. restart a service after its config changes).

Roles
A standardized directory structure for organizing playbooks, tasks, templates, and variables for reuse.

Vault
Encrypts sensitive data (passwords, keys) within Ansible files at rest.

Tags
Labels on tasks that let you selectively run/skip parts of a playbook.

5.2.2 Commands

```
ansible all -m ping # test connectivity to all hosts
ansible-playbook playbook.yml # run a playbook
ansible-inventory --list # view the inventory
ansible-vault create secrets.yml # create an encrypted secrets file
```

5.2.3 Practical Projects

- Install Nginx across 10 servers consistently
- Deploy an application and its dependencies
- Configure Docker on a fleet of hosts
- Patch servers on a schedule
- Create standard users and SSH keys
- Harden Linux servers to a security baseline (CIS-style)

Chapter 6

Containers: Docker & Kubernetes

6.1 Module 13: Docker Fundamentals

6.1.1 Core Concepts

Containers
Lightweight, isolated units that package an application with all its dependencies, sharing the host OS kernel.

Containers vs VMs
VMs virtualize hardware and each run a full guest OS (heavy, slow to boot). Containers virtualize at the OS level and share the host kernel (lightweight, start in seconds).

Docker Architecture
Client-server model: the Docker Client talks to the Docker Daemon (Engine), which builds/runs/manages images and containers.

Docker Daemon
The background service (`dockerd`) that does the actual work of building and running containers.

Docker Client
The CLI (`docker`) used to issue commands to the daemon.

Images
Read-only templates (layers) used to create containers.

Containers
Running (or stopped) instances of an image.

Registry & Docker Hub
A registry stores and distributes images; Docker Hub is the default public registry.

Layers
Each instruction in a Dockerfile creates a cached, reusable image layer – speeding up rebuilds.

6.1.2 Commands

```
docker pull # download an image from a registry
docker images # list local images
docker ps # list running containers (-a for all)
docker run # create and start a container from an image
docker exec # run a command inside a running container
```

```
docker logs # view container logs
docker inspect # show detailed JSON metadata about an object
docker stop # gracefully stop a running container
docker rm # remove a container
docker rmi # remove an image
docker build # build an image from a Dockerfile
```

6.2 Module 14: Docker Advanced

6.2.1 Dockerfile Instructions

FROM
Sets the base image to build on top of.

RUN
Executes a command at build time (e.g. installing packages), creating a new layer.

COPY
Copies files from the build context into the image.

ADD
Like COPY, but also supports remote URLs and auto-extracting tar archives (prefer COPY unless you need those features).

CMD
Default command executed when the container starts (can be overridden at runtime).

ENTRYPOINT
The fixed, main executable for the container; CMD often supplies its default arguments.

ENV
Sets environment variables available inside the container.

ARG
Build-time-only variables, not available in the running container.

EXPOSE
Documents which port the container listens on (does not actually publish it).

WORKDIR
Sets the working directory for subsequent instructions/commands.

USER
Sets the user the container process runs as – should not be root in production for security.

6.2.2 Advanced Topics

Multi-stage Builds
Use multiple FROM stages so build tools/dependencies don't bloat the final runtime image – smaller, more secure images.

Docker Networks
Virtual networks (bridge, host, overlay) that let containers communicate with each other and the outside world.

Volumes
Docker-managed persistent storage that survives container removal, independent of the container's writable layer.

Bind Mounts
Directly map a host directory/file into a container – useful for local development.

Docker Compose
Defines and runs multi-container applications from a single `docker-compose.yml` file.

Container Security
Run as non-root, use minimal base images, scan images for CVEs, drop unnecessary Linux capabilities, use read-only filesystems where possible.

6.2.3 Image Scanning Tools

- **Trivy** – fast, widely-used vulnerability & misconfiguration scanner for images, filesystems, and IaC.
- **Grype** – vulnerability scanner for container images and filesystems.
- **Syft** – generates a Software Bill of Materials (SBOM) from images/filesystems.

```
Source Code -> Dockerfile -> Image Build -> Security Scan -> Registry -> Container
```

6.3 Module 15: Kubernetes Fundamentals

Note
This is one of the largest and most important sections for a Cloud DevSecOps Engineer.

6.3.1 Architecture

```
Control Plane |- API Server (front door; all requests go through it) |- Scheduler
(assigns Pods to Nodes) |- Controller Manager (runs controllers that reconcile
desired state) |- etcd (key-value store holding cluster state)
Worker Node |- kubelet (agent that manages containers on the node) |- kube-proxy
(manages network rules for Services) |- Container Runtime (e.g.  containerd, runs the
actual containers)
```

6.3.2 Core Objects

Pods
The smallest deployable unit in Kubernetes – one or more tightly-coupled containers sharing network/storage.

ReplicaSets
Ensure a specified number of identical Pod replicas are running at all times.

Deployments
Manage ReplicaSets and provide declarative updates, rolling upgrades, and rollback for Pods.

Services
A stable network endpoint (virtual IP + DNS name) that routes traffic to a dynamic set of Pods.

Namespaces
Logical partitions within a cluster used to isolate and organize resources (e.g. per team/environment).

Labels & Selectors
Key-value tags attached to objects; selectors use labels to group/target resources (e.g. a Service selecting Pods).

6.3.3 Commands

```
kubectl get pods # list pods
kubectl get nodes # list cluster nodes
kubectl describe pod # detailed info + events for a pod
kubectl logs # view container logs in a pod
kubectl exec # run a command inside a container
kubectl apply -f file.yaml # create/update resources from a manifest
kubectl delete # remove a resource
```

6.4 Module 16: Kubernetes Advanced

6.4.1 Configuration & Storage

ConfigMaps
Store non-sensitive configuration data, injected into Pods as env vars or files.

Secrets
Like ConfigMaps but for sensitive data (base64-encoded by default – should be combined with encryption/Vault for real protection).

Volumes
Storage that a Pod's containers can mount, supporting many backend types.

PV (PersistentVolume)
A piece of storage provisioned in the cluster, independent of any Pod's lifecycle.

PVC (PersistentVolumeClaim)
A user's request for storage, which binds to a matching PV.

StorageClass
Defines a class of storage (e.g. SSD vs HDD) and enables dynamic provisioning of PVs.

6.4.2 Workload Types

StatefulSet
Manages Pods that need stable network identity and persistent storage (e.g. databases).

DaemonSet
Ensures a copy of a Pod runs on every (or selected) node – used for node-level agents like log collectors.

Job
Runs Pods to completion for a one-off task.

CronJob
Runs Jobs on a schedule, like cron.

6.4.3 Networking, Scaling & Scheduling

Ingress & Ingress Controller
Ingress defines HTTP(S) routing rules into the cluster; the Ingress Controller (e.g. Nginx) implements those rules.

HPA
Horizontal Pod Autoscaler – automatically scales the number of Pod replicas based on metrics like CPU.

VPA
Vertical Pod Autoscaler concepts – adjusts a Pod's CPU/memory requests automatically.

Resource Requests & Limits
Requests reserve minimum resources for scheduling; limits cap the maximum a container can use.

Liveness Probe
Checks if a container is still alive; restarts it if not.

Readiness Probe
Checks if a container is ready to receive traffic; removes it from Service endpoints if not.

Startup Probe
Gives slow-starting containers extra time before liveness/readiness checks kick in.

Rolling Updates & Rollback
Deployments update Pods gradually with zero downtime, and can revert to a previous revision if something breaks.

NodeSelector & Affinity/Anti-Affinity
Constrain which nodes Pods can be scheduled on, or keep/separate Pods from each other.

Taints & Tolerations
Taints repel Pods from a node unless the Pod has a matching toleration – used to dedicate nodes for specific workloads.

6.5 Module 17: Kubernetes Security

Note

This is a very important module for DevSecOps – expect deep interview questions here.

6.5.1 Access Control

RBAC
Role-Based Access Control – governs who (users/service accounts) can perform which actions on which Kubernetes resources.

Service Accounts
Identities used by Pods/processes (not humans) to authenticate to the Kubernetes API.

Roles / ClusterRoles
Roles grant permissions within a namespace; ClusterRoles grant permissions cluster-wide (or reusable across namespaces).

RoleBindings / ClusterRoleBindings
Attach a Role/ClusterRole to a specific user, group, or service account.

6.5.2 Workload & Network Security

Network Policies
Firewall rules for Pod-to-Pod traffic – deny by default, then explicitly allow required communication.

Pod Security Standards
Kubernetes-native policy levels (Privileged, Baseline, Restricted) defining what a Pod is allowed to do.

Security Context
Per-Pod/container settings controlling privilege, user ID, and Linux capabilities.

Run as Non-root
Prevents containers from running as UID 0, reducing the blast radius of a container compromise.

Read-only Root Filesystem
Prevents a compromised process from writing/modifying files inside the container.

Secrets
Should be encrypted at rest (etcd encryption) and access-restricted via RBAC.

Image Security
Only use scanned, trusted, minimal base images from a controlled registry.

Admission Control
Webhooks/policies (e.g. via Kyverno/OPA) that validate or mutate objects before they're admitted into the cluster.

Resource Limits
Prevent a single workload from exhausting node resources (a DoS vector).

Namespace Isolation
Uses namespaces plus RBAC/NetworkPolicy to segment tenants/teams within one cluster.

Audit Logging
Records every request made to the Kubernetes API server – essential for forensics and compliance.

Note

Official Kubernetes security guidance groups controls into two broad buckets: authentication/authorization (who can do what) and cluster/workload security (how workloads are isolated and hardened) – which maps closely to the RBAC and Network/Pod security topics above.

6.5.3 Security Tools

- **Trivy** – vulnerability scanning for images and Kubernetes manifests.
- **Kubescape** – scans clusters against security frameworks (NSA-CISA, MITRE ATT&CK).
- **kube-bench** – checks a cluster against the CIS Kubernetes Benchmark.
- **Falco** – runtime threat detection using syscall-level rules.

- **Kyverno** – Kubernetes-native policy engine (policies written as Kubernetes YAML).
- **OPA Gatekeeper** – policy engine using the Rego language for admission control.

6.6 Module 18: Helm

6.6.1 Core Concepts

Helm
The package manager for Kubernetes – bundles manifests into reusable, versioned, configurable packages.

Charts
A Helm package – a collection of templated Kubernetes manifests plus metadata.

Templates
Manifest files with placeholders that get filled in with values at install time.

values.yaml
The default configuration values consumed by chart templates (overridable per install).

Chart.yaml
Metadata file describing the chart (name, version, dependencies).

Repositories
Remote locations hosting Helm charts, analogous to a Docker registry.

Releases
A specific deployed instance of a chart in a cluster.

Upgrade & Rollback
Helm can update a release in place, and roll it back to a previous revision if needed.

6.6.2 Commands

```
helm install # deploy a chart as a new release
helm upgrade # update an existing release
helm rollback # revert a release to a previous revision
helm list # list releases
helm uninstall # remove a release
```

Chapter 7

CI/CD

7.1 Module 19: Jenkins

7.1.1 Architecture & Concepts

Jenkins Architecture
A Controller (master) coordinates builds; Agents (workers/nodes) execute the actual build/test/deploy steps, allowing distributed, parallel builds.

Jobs
A configured, runnable unit of work in Jenkins (freestyle job or pipeline).

Pipeline
A suite of automated steps (build, test, deploy) defined as code.

Declarative Pipeline
A structured, opinionated syntax (`pipeline { stages { ... } }`) – easier to read and lint.

Scripted Pipeline
A more flexible, Groovy-based syntax giving full programmatic control.

Jenkinsfile
The file (checked into source control) that defines a pipeline as code.

Plugins
Extend Jenkins with integrations (Git, Docker, Slack, security scanners, etc.).

Credentials
Securely stored secrets (tokens, SSH keys, passwords) that pipelines can reference without exposing them in code.

Parameters
Let a pipeline accept input values at trigger time (e.g. environment name).

Webhooks
Allow external systems (e.g. GitHub) to automatically trigger a Jenkins build on events like a push.

Shared Libraries
Reusable Groovy code shared across multiple Jenkinsfiles to avoid duplication.

7.1.2 Typical Secure Pipeline

```
Checkout -> Build -> Unit Test -> SAST -> SCA -> Docker Build
-> Image Scan -> Push Registry -> Deploy -> DAST
```

7.2 Module 20: GitHub Actions / GitLab CI

7.2.1 GitHub Actions

Workflow
The top-level automation definition (a YAML file under `.github/workflows`), triggered by events.

Jobs
A set of steps that run on the same runner; jobs can run in parallel or depend on each other.

Steps
Individual tasks within a job (run a command, or use an Action).

Actions
Reusable, shareable units of automation (from the Marketplace or custom).

Runner
The machine (GitHub-hosted or self-hosted) that executes jobs.

Secrets & Variables
Encrypted values (Secrets) or plain configuration (Variables) available to workflows.

Environments
Named deployment targets (e.g. staging, production) that can require manual approval and hold environment-specific secrets.

Artifacts
Files produced by a job that can be shared with later jobs or downloaded after the run.

Matrix Builds
Run the same job across multiple combinations of inputs (e.g. OS × language version) in parallel.

OIDC Authentication
Lets workflows exchange a short-lived OIDC token for cloud credentials (e.g. AWS role) instead of storing long-lived cloud secrets.

```
name: CI

on:
  push:

jobs:
  build:
    runs-on: ubuntu-latest
```

7.2.2 GitLab CI/CD

.gitlab-ci.yml
The pipeline definition file at the repository root.

GitLab Runner
The agent that executes CI/CD jobs defined in `.gitlab-ci.yml`.

Stages
Logical groups of jobs (e.g. build, test, deploy) that run in order; jobs within a stage run in parallel.

Jobs
Individual tasks within a stage.

Artifacts
Files passed between stages/jobs, similar to GitHub Actions artifacts.

Chapter 8

Core DevSecOps: Pipeline Security & Testing

8.1 Module 21: DevSecOps Pipeline Security

Note

OWASP describes DevSecOps as integrating automated security checks directly into the existing CI/CD process – rather than running security as a separate, blocking phase at the end.

8.1.1 Full Secure Pipeline

```
Developer -> Git -> Secret Scan -> SAST -> Build -> SCA -> Docker Build
-> Container Scan -> Registry -> Kubernetes -> DAST -> Runtime Security
-> Monitoring
```

8.2 Module 22: SAST – Static Application Security Testing

Analyzes source code (without running it) to find security vulnerabilities early, directly in the developer's workflow.

8.2.1 Tools

- **SonarQube** – continuous code quality & security analysis platform with Quality Gates.
- **Semgrep** – fast, rule-based static analysis supporting custom security rules.
- **CodeQL** – GitHub's semantic code analysis engine, treats code as queryable data.
- **Checkmarx / Fortify (concepts)** – enterprise-grade commercial SAST platforms.

8.2.2 Key Terms

Vulnerability
A weakness that could be exploited to compromise confidentiality, integrity, or availability.

Bug
A functional defect in code (not necessarily a security issue).

Code Smell
A pattern indicating poor code quality/maintainability, not a direct security risk.

Security Hotspot
Code that needs manual review to determine if it's actually exploitable.

Quality Gate
A defined set of conditions (coverage, bugs, vulnerabilities) a build must pass to be considered releasable.

False Positive
A finding flagged as an issue that, on review, is not actually a real problem.

8.3 Module 23: SCA – Software Composition Analysis

Scans your project's open-source dependencies for known vulnerabilities and license risk.

8.3.1 Key Terms

Open-source Dependencies
Third-party libraries/packages your application relies on.

CVE
Common Vulnerabilities and Exposures – a standardized public identifier for a known vulnerability.

Package Vulnerabilities
Known security flaws in a specific version of a dependency.

Dependency Scanning
Automated checking of your dependency tree against vulnerability databases.

License Risks
Some open-source licenses (e.g. copyleft) impose obligations that may conflict with your product's licensing.

Transitive Dependencies
Dependencies pulled in indirectly by your direct dependencies – often the source of hidden vulnerabilities.

8.3.2 Tools

- **Snyk** – developer-first SCA and container scanning platform.
- **OWASP Dependency-Check** – open-source dependency vulnerability scanner.
- **Trivy** – also scans dependencies (in addition to images/IaC).
- **Dependabot** – automatically opens PRs to upgrade vulnerable dependencies on GitHub.

8.4 Module 24: DAST – Dynamic Application Security Testing

Tests a *running* application from the outside (like an attacker would) to find exploitable vulnerabilities that only appear at runtime.

8.4.1 Tools

- **OWASP ZAP** – open-source DAST scanner/proxy for web applications.
- **Burp Suite (concepts)** – industry-standard interception proxy for manual and automated web app security testing.

8.4.2 What DAST Finds

SQL Injection
Attacker-supplied input alters a database query, potentially exposing or modifying data.

XSS (Cross-Site Scripting)
Attacker-supplied script runs in another user's browser via unsanitized output.

Authentication Issues
Weak login flows, missing lockouts, or broken password reset logic.

Session Issues
Predictable, non-expiring, or improperly protected session tokens/cookies.

Security Headers
Missing headers (CSP, HSTS, X-Frame-Options) that would otherwise mitigate common attacks.

Chapter 9

Secrets & Supply Chain Security

9.1 Module 25: Secrets Security

Note

Never store the following directly in Git: passwords, API keys, AWS access keys, database credentials, tokens, or certificates. Once committed, they remain in Git history even after deletion unless the history itself is rewritten.

9.1.1 Secret Management Platforms

HashiCorp Vault
A centralized secrets-management platform supporting dynamic secrets, encryption-as-a-service, and fine-grained access policies.

AWS Secrets Manager
Managed secret storage with automatic rotation for supported services (RDS, etc.).

AWS Parameter Store
Simpler key-value configuration/secret storage (part of Systems Manager).

Kubernetes Secrets
Cluster-native secret objects – base64-encoded by default, should be paired with encryption at rest and RBAC.

GitHub Secrets
Encrypted secrets scoped to a repo/org/environment, injected into GitHub Actions workflows.

Secret Rotation
Periodically and automatically replacing credentials to limit the impact of any leaked secret.

9.1.2 Secret Scanning Tools

- **Gitleaks** – scans git repositories/history for hardcoded secrets.
- **TruffleHog** – deep-scans repositories (including history) for high-entropy strings and known secret patterns.

9.2 Module 26: Software Supply Chain Security

Note

Supply chain security has become one of the most important areas in modern DevSecOps, following major real-world incidents (e.g. SolarWinds) where attackers compromised the build pipeline itself, not just the running application.

9.2.1 Key Concepts

SBOM
Software Bill of Materials – a complete inventory of every component/dependency in a piece of software, enabling fast vulnerability response.

Software Supply Chain
Everything involved in producing software – source code, dependencies, build tools, CI/CD infrastructure, and distribution.

Dependency Provenance
Verifiable evidence of where a dependency/artifact actually came from and how it was built.

Artifact Integrity
Ensuring a built artifact hasn't been tampered with between build and deployment.

Image Signing / Package Signing
Cryptographically signing artifacts so consumers can verify authenticity and integrity before use.

Vulnerability Management
The ongoing process of identifying, prioritizing, and remediating vulnerabilities across the software estate.

9.2.2 Tools & Standards

- **Syft** – generates SBOMs from source/images.
- **Grype** – scans SBOMs/images for known vulnerabilities.
- **Cosign** – signs and verifies container images and artifacts.
- **Sigstore** – an open ecosystem (including Cosign) for signing, verifying, and proving provenance of software.
- **SLSA** – Supply chain Levels for Software Artifacts, a framework/maturity model for build integrity.
- **CycloneDX & SPDX** – the two leading standard SBOM formats.

```
Code -> Dependencies -> Build -> SBOM -> Sign Artifact -> Scan -> Deploy -> Verify
```

Chapter 10

Cloud Security & Policy as Code

10.1 Module 27: Cloud Security

10.1.1 Core Controls

IAM
The foundation of cloud security – who can do what, enforced through users, roles, and policies.

Least Privilege
Grant the minimum permissions necessary, reviewed and tightened continuously.

RBAC
Access decisions based on a user/service's assigned role rather than individual grants.

MFA
An essential control that stops most credential-based account takeovers.

Encryption at Rest / in Transit
Data is encrypted while stored (disk, database, object storage) and while moving across networks (TLS).

KMS & Key Rotation
Centralized key management with periodic automatic rotation limits the blast radius of a compromised key.

Network Segmentation
Dividing a network into isolated zones (public/private subnets, security groups) to limit lateral movement.

WAF & DDoS Protection
Filters malicious application-layer traffic and absorbs/mitigates volumetric attacks.

Logging & Auditing
Centralized, tamper-evident logs (CloudTrail, VPC Flow Logs) are essential for detection and forensics.

Cloud Security Posture
The overall state of an environment's configuration against security best practices – continuously monitored (CSPM tools).

10.1.2 Guiding Principles

Zero Trust
Never implicitly trust any user, device, or network – verify every request explicitly, regardless of origin.

Defense in Depth
Layer multiple independent security controls so a single failure doesn't lead to full compromise.

Principle of Least Privilege
Only grant what's needed, for only as long as it's needed.

Shared Responsibility
Cloud provider secures the underlying platform; the customer secures their configuration, data, and application.

Identity Federation
Allows users to authenticate using an existing trusted identity provider (e.g. corporate SSO) instead of separate cloud credentials.

Temporary Credentials
Short-lived credentials (via STS/roles) that automatically expire, reducing risk if leaked.

10.2 Module 28: Policy as Code

Encodes security and compliance rules as machine-readable, version-controlled code, so policy is automatically enforced instead of relying on manual review.

10.2.1 Frameworks

OPA
Open Policy Agent – a general-purpose policy engine usable across Kubernetes, APIs, and infrastructure.

Rego
The declarative query language used to write OPA policies.

Gatekeeper
The Kubernetes admission-controller integration for OPA.

Kyverno
A Kubernetes-native policy engine that writes policies as plain Kubernetes YAML (no new language to learn).

Terraform Policy Checking
Evaluating Terraform/OpenTofu plans against policy rules before they're allowed to apply.

10.2.2 Security Scanning Tools for IaC

- **Checkov** – static analysis for Terraform, CloudFormation, Kubernetes, and more.
- **tfsec** – security scanner specifically for Terraform code.
- **Terrascan** – policy-as-code scanner for IaC across multiple providers.

10.2.3 Example Policies

- No public S3 buckets
- No security group rule allowing SSH from 0.0.0.0/0
- Encryption mandatory on all storage resources
- Only approved/whitelisted instance types allowed
- Containers must not run as root

Chapter 11

Observability: Monitoring, Logging & Tracing

11.1 Module 29: Monitoring

11.1.1 Core Concepts

Infrastructure Monitoring
Tracking the health of servers, networks, and hardware-level resources.

Application Monitoring
Tracking the performance and behavior of running applications (response times, error rates).

Metrics
Numeric measurements over time (CPU %, request count, latency).

Logs
Timestamped, discrete event records emitted by systems/applications.

Traces
Records of a single request's journey across multiple services, showing where time was spent.

Alerts
Automated notifications triggered when a metric crosses a defined threshold.

Dashboards
Visual, real-time views of metrics/logs for humans to monitor system health.

11.1.2 Tools

Prometheus
An open-source time-series metrics database that scrapes (pulls) metrics from targets.

Grafana
A visualization layer that builds dashboards on top of Prometheus (and many other data sources).

Alertmanager
Handles alerts sent by Prometheus – deduplicates, groups, and routes them to Slack/Email/-PagerDuty.

AWS CloudWatch
AWS's native metrics, logs, and alarms service.

PromQL
Prometheus's query language for selecting and aggregating time-series data – worth learning the basics (e.g. `rate()`, `sum by()`).

```
Application -> Metrics -> Prometheus -> Grafana -> Alertmanager
-> Slack / Email / Pager
```

11.2 Module 30: Centralized Logging

11.2.1 Stack Components

Elasticsearch / OpenSearch
A distributed search-and-analytics engine used to index and query large volumes of log data.

Logstash
A data-processing pipeline that ingests, transforms, and ships logs.

Fluent Bit / Fluentd
Lightweight log collectors/forwarders, commonly run as DaemonSets in Kubernetes to ship container logs.

Kibana
The visualization/UI layer for Elasticsearch/OpenSearch.

```
Application Logs -> Fluent Bit -> OpenSearch -> Dashboards
```

Note

Loki / Grafana Loki is a modern, cost-efficient alternative: it indexes only log metadata (labels) instead of full text, pairing naturally with Grafana for a unified metrics+logs view.

11.3 Module 31: OpenTelemetry

A vendor-neutral standard (and toolset) for generating and collecting telemetry data uniformly across an application stack.

11.3.1 Core Concepts

Metrics, Logs, Traces
The three pillars of observability, all captured under one unified OpenTelemetry standard.

Distributed Tracing
Following a single request as it flows through multiple microservices.

Spans
An individual unit of work within a trace (e.g. one service call).

Trace IDs
A unique identifier correlating all spans that belong to the same end-to-end request.

OpenTelemetry Collector
A vendor-agnostic component that receives, processes, and exports telemetry data to backends like Prometheus or a tracing system.

```
Application -> OpenTelemetry -> Collector -> Prometheus / Grafana /
Tracing Backend
```

11.4 Module 32: SIEM & Runtime Security

11.4.1 Core Concepts

SIEM
Security Information and Event Management – aggregates and correlates security events from across the environment to detect threats.

SOC
Security Operations Center – the team/function responsible for monitoring, detecting, and responding to security incidents.

Security Events
Any recorded occurrence relevant to security (login, permission change, network connection).

Incident Detection
Identifying that a security event or pattern of events indicates a real incident.

Runtime Threat Detection
Monitoring running workloads (not just static code/images) for malicious behavior in real time.

11.4.2 Tools

- **Splunk** – widely-used commercial SIEM/log-analytics platform.
- **Elastic Security** – SIEM built on the Elastic Stack.
- **AWS Security Hub & GuardDuty** – AWS-native aggregated findings and threat detection.
- **Falco** – open-source runtime security for containers/Kubernetes using syscall rules.

11.4.3 Example Detections

- Suspicious login (unusual location/time/device)
- Privilege escalation attempts
- Unexpected shell spawned inside a container
- Crypto-mining process signatures
- Unauthorized/anomalous API calls
- Malicious or drifted container behavior

Chapter 12

Compliance & Reliability

12.1 Module 33: Governance, Risk & Compliance (GRC)

12.1.1 Key Frameworks (at least basics)

SOC 2
An auditing standard evaluating a service organization's controls around security, availability, and confidentiality.

ISO 27001
An international standard for building and operating an Information Security Management System (ISMS).

PCI DSS
Payment Card Industry Data Security Standard – required for anyone storing/processing/transmitting card data.

HIPAA
U.S. regulation governing the privacy and security of health information.

GDPR
EU regulation governing the protection and privacy of personal data.

NIST
A U.S. framework (e.g. NIST CSF, 800-53) widely used as a reference for security controls.

CIS Benchmarks
Prescriptive, consensus-based hardening configuration standards for OS, cloud, and Kubernetes.

12.1.2 Core GRC Terms

Policy
A documented statement of intent/rules an organization commits to follow.

Control
A specific safeguard implemented to meet a policy requirement (technical or procedural).

Risk
The potential for loss/harm resulting from a threat exploiting a vulnerability.

Audit
An independent review verifying that controls are actually implemented and effective.

Evidence
Artifacts (logs, screenshots, configs) proving a control is operating as claimed.

Compliance
The state of meeting all required policies/regulations/standards.

Exception
A documented, approved deviation from a policy/control, usually with compensating measures.

Remediation
The process of fixing an identified gap or finding.

12.2 Module 34: SRE Concepts

Note

Site Reliability Engineering complements DevSecOps by applying engineering discipline to operational reliability.

12.2.1 Core Terms

SLA
Service Level Agreement – an external, often contractual, commitment about service performance.

SLO
Service Level Objective – an internal target for a specific reliability metric (e.g. 99.9% uptime).

SLI
Service Level Indicator – the actual measured metric used to evaluate an SLO (e.g. request success rate).

Error Budget
The allowable amount of unreliability (derived from the SLO) that can be "spent" before releases are slowed to focus on stability.

Availability & Reliability
Availability is whether the system is up; reliability is whether it consistently performs correctly.

MTTR
Mean Time To Recovery – average time taken to restore service after an incident.

MTBF
Mean Time Between Failures – average time between one failure and the next.

Incident Management
The structured process of detecting, responding to, and resolving incidents.

Postmortem
A blameless written review after an incident, capturing root cause and follow-up actions.

Root Cause Analysis (RCA)
The systematic process of identifying the true underlying cause of an incident, not just its symptoms.

On-call
A rotation of engineers responsible for responding to alerts/incidents outside normal hours.

Runbooks / Playbooks
Documented, step-by-step procedures for handling known operational or incident scenarios.

Chapter 13

Deployment Strategies & GitOps

13.1 Module 35: Deployment Strategies

Strategy	Advantages	Disadvantages	Typical Use
Recreate	Simple; no version overlap	Downtime during switch	Non-critical/internal apps
Rolling	No downtime; gradual rollout	Old & new versions run briefly together	Default for most services
Blue-Green	Instant cutover & rollback	Requires double the infrastructure briefly	Critical apps needing instant rollback
Canary	Very low risk; real-user validation	More complex to set up/-monitor	High-traffic, risk-sensitive releases
A/B	Tests business/UX outcomes, not just stability	Needs analytics tooling, longer running	Feature/UX experiments
Feature Flags	Decouples deploy from release; instant toggle	Flag debt if not cleaned up	Gradual feature rollout, kill-switches

Rollback Strategy

For each deployment strategy, know exactly how to revert – e.g. Rolling: roll back the Deployment revision; Blue-Green: flip traffic back to the old (blue) environment; Canary: route 100% traffic back to the stable version.

13.2 Module 36: GitOps

Git is the single source of truth for both application and infrastructure desired state; an automated controller continuously reconciles the live environment to match Git.

13.2.1 Tools

- **Argo CD** – a declarative, Kubernetes-native GitOps continuous delivery tool.
- **Flux** – another popular GitOps operator for Kubernetes, tightly integrated with Kubernetes CRDs.

```
Developer -> Git -> Manifest / Helm -> Argo CD -> Kubernetes
```

13.2.2 Key Concepts

Desired State
The configuration declared in Git – the "source of truth."

Reconciliation
The continuous process of comparing live cluster state to the desired state in Git and correcting drift.

Auto Sync
The GitOps controller automatically applies changes as soon as Git changes.

Manual Sync
Changes are staged but require explicit approval before being applied.

Drift Detection
Identifying when the live cluster no longer matches what's declared in Git (e.g. due to a manual `kubectl edit`).

Rollback
Reverting Git to a previous commit automatically reverts the live environment too.

Chapter 14

Managed Kubernetes, Serverless & Data

14.1 Module 37: AWS EKS

14.1.1 Detailed Concepts

EKS Control Plane
The Kubernetes API server, etcd, and scheduler – fully managed and made highly available by AWS.

Worker Nodes
EC2 instances (or Fargate) that actually run your Pods.

Managed Node Groups
AWS-managed EC2 Auto Scaling Groups for worker nodes, simplifying provisioning/upgrades/scaling.

Fargate
Serverless compute for EKS – run Pods without managing any underlying EC2 nodes.

IAM Roles for Service Accounts (IRSA)
Lets individual Kubernetes Pods assume fine-grained AWS IAM roles instead of sharing the node's IAM role – a key least-privilege pattern.

VPC CNI
The networking plugin that gives each Pod a real VPC IP address, enabling native AWS networking integration.

EBS CSI Driver
Enables Kubernetes to dynamically provision and attach EBS volumes as persistent storage.

AWS Load Balancer Controller
Automatically provisions ALBs/NLBs from Kubernetes Ingress/Service objects.

Cluster Autoscaler
Automatically adds/removes worker nodes based on Pod scheduling pressure.

ECR Integration
Pulls container images directly from AWS's managed registry.

CloudWatch & Secrets Integration
EKS can ship logs/metrics to CloudWatch and pull secrets from Secrets Manager/Parameter

Store.

14.2 Module 38: Serverless DevSecOps

14.2.1 AWS Serverless Building Blocks

Lambda
Runs code in response to events without provisioning servers; billed per invocation/duration.

API Gateway
Manages, secures, and exposes APIs that front Lambda (or other backends).

EventBridge
A serverless event bus for routing events between AWS services and applications.

SQS
Simple Queue Service – managed message queuing for decoupling components.

SNS
Simple Notification Service – managed pub/sub messaging.

Step Functions
Orchestrates multiple Lambda functions/services into a visual, stateful workflow.

14.2.2 Serverless Security

- Scope each function's IAM execution role to the absolute minimum permissions it needs.
- Pull secrets from Secrets Manager/Parameter Store at runtime – never hardcode them.
- Scan function dependencies for vulnerabilities just like any other code.
- Enable structured logging and forward it to CloudWatch for visibility.
- Enable CloudTrail to capture every API call the function's role makes, for audit purposes.

14.3 Module 39: Database for DevSecOps

Note

DBA-level depth isn't required, but you must understand the operational and security fundamentals of the databases your pipelines deploy and operate.

14.3.1 Engine Types

SQL – MySQL / PostgreSQL
Relational databases using structured schemas and SQL queries; strong consistency (ACID).

NoSQL – DynamoDB
Managed, schema-less, key-value/document store with near-infinite horizontal scale.

NoSQL – MongoDB (concepts)
A popular document-oriented NoSQL database.

Caching – Redis
An in-memory data store used for caching, session storage, and pub/sub messaging.

14.3.2 Operational Concepts

Backup
Regularly saved, restorable copies of the database.

Replication
Copying data to one or more secondary instances for redundancy and read scaling.

High Availability
Automatic failover to a standby if the primary database fails.

Encryption
Protecting data at rest and in transit for compliance and security.

Database Security
Network isolation (private subnets), least-privilege DB users, and audit logging.

Connection Pools
Reuse a limited set of open DB connections across requests instead of opening a new one each time – critical for performance at scale.

Credentials
Should be pulled dynamically from a secrets manager, never hardcoded, and rotated regularly.

Read Replica
A read-only copy of the database used to offload read traffic from the primary.

Multi-AZ
Deploying a database (and its standby) across multiple Availability Zones for resilience against a data-center failure.

14.4 Module 40: Application Architecture

```
User -> DNS -> CDN -> WAF -> Load Balancer -> Frontend -> Backend
-> Cache -> Database
```

14.4.1 Architectural Patterns

Monolith
A single, tightly-coupled codebase/deployment unit – simple to start, harder to scale/deploy independently.

Microservices
Independently deployable services, each owning a specific business capability – more operational complexity but better scalability/team autonomy.

REST APIs
The dominant style for stateless, resource-oriented HTTP APIs using standard verbs (GET/-POST/PUT/DELETE).

API Gateway
A single entry point that routes, authenticates, rate-limits, and monitors API traffic to backend services.

Message Queue
Decouples producers and consumers by buffering messages between them (e.g. SQS, RabbitMQ).

Event-Driven Architecture
Services communicate by publishing/reacting to events rather than direct synchronous calls.

Kafka (basics)
A distributed event-streaming platform for high-throughput, durable, ordered event pipelines.

RabbitMQ (basics)
A traditional message broker supporting flexible routing (queues, exchanges) for asynchronous messaging.

Chapter 15

Production Troubleshooting & Incident Handling

15.1 Module 41: Production Troubleshooting

Note

This is one of the most heavily interviewed areas. For every scenario, structure your answer as: **Problem → Possible Causes → Commands/Logs → Investigation → Root Cause → Fix → Prevention.**

15.1.1 General Infrastructure Scenarios

Server unreachable
Check network path (`ping`, `traceroute`), security group/firewall rules, instance status checks, and whether the OS itself has hung or run out of resources.

CPU at 100%
Use `top`/`htop` to find the offending process; check for runaway loops, insufficient instance sizing, or a traffic spike; consider scaling out or fixing the code.

Memory high
Use `free`/`top` to identify the consumer; check for memory leaks, misconfigured JVM/heap settings, or genuinely undersized instances.

Disk full
Use `df -h` and `du -sh` to find what's consuming space (often logs); clean up, rotate logs, or expand the volume; set up disk-usage alerting to prevent recurrence.

Application down
Check the process/service status (`systemctl status`), application logs, recent deployments, and dependent services (DB, cache).

15.1.2 Web & DNS/SSL Scenarios

502 Bad Gateway
The load balancer/reverse proxy got an invalid response from the backend – check backend health, backend crash logs, and timeouts.

503 Service Unavailable
The backend is overloaded or intentionally down (e.g. no healthy targets) – check target group

health checks and Auto Scaling capacity.

DNS issue
Verify records with `dig`/`nslookup`, check TTLs, propagation, and whether the correct hosted zone/record is being queried.

SSL issue
Check certificate expiry, certificate chain completeness, the domain matching the cert (SAN/CN), and TLS version/cipher mismatches.

15.1.3 Container & Kubernetes Scenarios

Docker container crashing
Check `docker logs`, exit code, resource limits, and whether the entrypoint/command is correct.

Pod CrashLoopBackOff
The container keeps starting and crashing – check `kubectl logs` (and `-previous`), `kubectl describe pod` for events, application startup errors, or a bad readiness/liveness probe config.

ImagePullBackOff
The node can't pull the image – check the image name/tag, registry credentials (imagePullSecrets), and registry network reachability.

Pending pod
The scheduler can't place the Pod – check for insufficient node resources, unsatisfied node selectors/affinity, or missing PVCs.

OOMKilled
The container exceeded its memory limit and was killed – check actual memory usage vs the configured `limits.memory`, and raise the limit or fix a leak.

Kubernetes service unreachable
Check Service selector matches Pod labels, Endpoints are populated, NetworkPolicy isn't blocking traffic, and DNS (CoreDNS) resolution.

15.1.4 Pipeline & Cloud Scenarios

Jenkins build failed
Check the console log for the exact failing stage, credential issues, dependency resolution errors, or a flaky test.

Terraform state locked
Someone else's apply may still be running, or a previous run crashed without releasing the lock – verify no one is actively applying, then release the lock (e.g. `terraform force-unlock`) carefully.

AWS permission denied
Read the exact IAM error (it names the missing action/resource), then check the identity's attached policies, permission boundaries, and any SCPs blocking it.

Database connectivity issue
Check security groups/NACLs, database status, connection pool exhaustion, correct endpoint/credentials, and whether the app is in the right subnet/VPC.

Slow application
Profile where time is spent – database query latency, external API calls, insufficient caching, network hops, or under-provisioned compute.

15.2 Module 42: Security Incident Handling

```
Detect -> Alert -> Investigate -> Contain -> Remediate -> Recover -> Postmortem
```

15.2.1 Example Incidents & Response Focus

Leaked AWS Key
Immediately deactivate/rotate the key, review CloudTrail for unauthorized usage, and add secret scanning to prevent recurrence.

Public S3 Bucket
Restrict the bucket policy/Block Public Access settings immediately, audit what was exposed, and add Config rules to detect this automatically going forward.

Vulnerable Docker Image
Patch/rebuild from an updated base image, redeploy, and add image scanning as a mandatory pipeline gate.

Root Container
Rebuild the image to run as a non-root user and enforce this via Pod Security Standards/admission policy.

Unauthorized IAM Access
Revoke the credentials/role, review CloudTrail for actions taken, and tighten the associated policy to least privilege.

Kubernetes Compromised Pod
Isolate the Pod (NetworkPolicy/cordon node), capture forensic data, terminate/redeploy clean, and investigate the entry vector.

Secret Accidentally Committed to Git
Rotate the secret immediately (assume it's compromised), then scrub it from Git history and add pre-commit secret scanning.

Chapter 16

High Availability, DR & Cost Optimization

16.1 Module 43: High Availability Architecture

```
Route53 | CloudFront | WAF | ALB | EKS / EC2 /
AZ-1 AZ-2 | RDS Multi-AZ
```

16.1.1 Core Concepts Recap

HA
Architecting so that the failure of any single component doesn't take down the whole system.

Scalability
Add capacity (horizontally or vertically) as demand grows.

Resilience
The system's ability to withstand and recover from failures/disruptions gracefully.

Fault Tolerance
Continuing correct operation even while a component has failed.

Backup
A safety net independent of the live system, used to restore data/state.

DR (Disaster Recovery)
The broader plan and capability to recover an entire system/region after a catastrophic event.

16.2 Module 44: Backup & Disaster Recovery

16.2.1 Core Terms

Backup
A saved copy of data at a point in time.

Snapshot
A backup of an entire volume/disk state, often incremental.

Restore
The process of recovering data/systems from a backup or snapshot.

RPO
Maximum tolerable data loss, expressed as time (drives how frequently you back up).

RTO
Maximum tolerable downtime before service must be restored (drives your DR strategy choice).

16.2.2 DR Strategies (increasing cost, decreasing RTO)

Backup & Restore
Cheapest – data is backed up regularly; in a disaster, infrastructure is rebuilt and data restored from backup. Slowest RTO.

Pilot Light
A minimal version of the environment (e.g. just the database) always running in the DR region, scaled up only when needed.

Warm Standby
A scaled-down but fully functional copy of the full environment always running, ready to be scaled up quickly.

Active-Active
Full production capacity running simultaneously in two (or more) regions, handling live traffic – fastest RTO/RPO, highest cost.

16.3 Module 45: FinOps Basics

Note

A well-rounded Cloud DevSecOps Engineer also understands cost – unmonitored cloud spend is a common real-world failure mode.

16.3.1 Key Tools & Levers

AWS Cost Explorer
Visualizes and analyzes historical and forecasted AWS spend.

Budgets
Set spend thresholds and get alerted before/when they're exceeded.

Reserved Instances
Commit to 1–3 years of usage in exchange for a significant discount over on-demand pricing.

Savings Plans
A flexible commitment-based discount model (spend-based rather than instance-type-based).

Spot Instances
Spare AWS capacity at steep discounts, usable for fault-tolerant/interruptible workloads.

Rightsizing
Matching instance/resource sizes to actual observed usage, avoiding over-provisioning.

Storage Lifecycle
Automatically moving older/less-accessed data to cheaper storage tiers.

Kubernetes Resource Optimization
Tuning requests/limits and using autoscaling to avoid paying for idle cluster capacity.

Tagging & Cost Allocation
Consistently tagging resources (team, project, environment) so spend can be attributed and analyzed accurately.

Chapter 17

Certificates, Identity & Kubernetes Networking

17.1 Module 46: SSL/TLS

17.1.1 Core Concepts

HTTP vs HTTPS
HTTPS wraps HTTP in TLS encryption, protecting data in transit and authenticating the server.

SSL/TLS
The cryptographic protocol suite that provides that encryption and authentication (TLS is the current, secure standard).

Certificate
A digitally-signed document binding a public key to an identity (e.g. a domain name).

CA (Certificate Authority)
A trusted entity that issues and signs certificates, vouching for their authenticity.

Public/Private Key
The certificate's public key is shared with everyone; the private key is kept secret and used to prove ownership/decrypt.

Encryption
Protects the confidentiality of data transmitted over the connection.

Certificate Chain
The path from a server's certificate up through intermediate CAs to a trusted root CA.

CSR
Certificate Signing Request – generated by the applicant and submitted to a CA to obtain a signed certificate.

Certificate Expiration
Certificates are time-limited; an expired certificate breaks trust and must be renewed/rotated before it lapses.

17.1.2 Tools & Services

- **OpenSSL** – the standard CLI toolkit for generating keys, CSRs, and inspecting certificates.

- **AWS ACM** – AWS Certificate Manager, provisions and auto-renews certificates for use with ALB/CloudFront.
- **Let's Encrypt** – a free, automated CA widely used for public-facing certificates.
- **cert-manager** – automates certificate issuance and renewal inside Kubernetes clusters.

17.2 Module 47: Authentication & Authorization

17.2.1 Core Concepts

Authentication (AuthN)
Verifying *who* you are (login, credentials).

Authorization (AuthZ)
Determining *what* you're allowed to do once authenticated.

IAM
The cloud-provider system implementing both authentication and authorization for cloud resources.

RBAC
Assigning permissions based on a user's role rather than individually – used in both cloud IAM and Kubernetes.

OAuth 2.0
An authorization framework letting an application access resources on a user's behalf without handling their password directly.

OIDC
OpenID Connect – an identity layer built on top of OAuth 2.0, adding standardized authentication (who the user is).

JWT
JSON Web Token – a compact, signed token format commonly used to carry identity/claims between parties (e.g. after OIDC login).

SSO
Single Sign-On – log in once and gain access to multiple connected applications/services.

SAML
An older XML-based standard for exchanging authentication/authorization data, common in enterprise SSO.

MFA
Requiring a second factor beyond a password, drastically reducing account-takeover risk.

Note

This module is extremely high-value – it underpins cloud IAM, Kubernetes RBAC, and CI/CD pipeline authentication (e.g. OIDC-based cloud role assumption from GitHub Actions).

17.3 Module 48: Kubernetes Networking

17.3.1 Traffic Patterns

Pod-to-Pod
Every Pod gets its own IP and can communicate directly with any other Pod's IP by default (flat network model), unless restricted by NetworkPolicy.

Pod-to-Service
Traffic to a Service's virtual IP is load-balanced across the healthy Pods backing it.

Service-to-Service
Services typically discover each other via cluster-internal DNS names rather than hardcoded IPs.

17.3.2 Service Types & Components

ClusterIP
The default Service type – exposes the Service only on an internal cluster IP, not reachable from outside.

NodePort
Exposes the Service on a static port on every node's IP, reachable from outside the cluster.

LoadBalancer
Provisions an external cloud load balancer (e.g. ALB/NLB) pointing at the Service.

Ingress
Provides HTTP(S) routing (host/path based) into the cluster, typically fronted by a single external load balancer for many services.

CNI
Container Network Interface – the plugin standard that actually implements Pod networking.

CoreDNS
The default DNS server inside a Kubernetes cluster, resolving Service/Pod names.

NetworkPolicy
Kubernetes-native firewall rules controlling which Pods can talk to which.

17.3.3 Common CNIs

- **AWS VPC CNI** – gives Pods real VPC IP addresses (used by default on EKS).
- **Calico** – widely-used CNI with strong NetworkPolicy enforcement.
- **Cilium (concepts)** – eBPF-based CNI offering high-performance networking, deep observability, and advanced security policies.

17.4 Module 49: Service Mesh

17.4.1 Core Concepts

Istio
A popular open-source service mesh that adds traffic management, security, and observability to microservices without changing application code.

Envoy
The high-performance proxy that Istio (and many meshes) deploys as a sidecar next to each service instance.

Service-to-Service Communication
The mesh intercepts and manages all traffic between services transparently via sidecar proxies.

mTLS
Mutual TLS – both client and server authenticate each other, automatically encrypting all service-to-service traffic within the mesh.

Traffic Management
Fine-grained control over routing – e.g. splitting traffic by percentage, header-based routing.

Observability
The mesh automatically generates rich metrics, logs, and traces for all inter-service traffic.

Retry & Circuit Breaking
The mesh can automatically retry failed calls and stop sending traffic to an unhealthy service (circuit breaking) to prevent cascading failures.

Canary Routing
Gradually shifting a small percentage of traffic to a new service version to validate it safely.

Chapter 18

Core Principles, Capstone Project & Interview Prep

18.1 Module 50: Core Security Concepts

18.1.1 Must-Remember Principles

CIA Triad
The three foundational goals of security: **Confidentiality** (only authorized parties can see data), **Integrity** (data can't be tampered with undetected), **Availability** (systems stay accessible when needed).

Least Privilege
Grant only the minimum access required, nothing more.

Defense in Depth
Use multiple overlapping layers of security controls, so no single failure is catastrophic.

Zero Trust
Verify every request explicitly – never trust based on network location alone.

Separation of Duties
No single person/system should have unchecked end-to-end control over a sensitive process (reduces fraud/error risk).

Secure by Default
Systems should ship with the safest configuration out of the box, requiring deliberate action to loosen security.

Shift Left
Catch security issues as early as possible in the development lifecycle (code/commit time).

Shift Right
Continue security monitoring and testing after release, in the live production environment.

18.2 Module 51: End-to-End DevSecOps Project

Note

This single project ties together nearly the entire Cloud DevSecOps journey covered in this document – it is the best thing to describe in a system-design/architecture interview.

18.2.1 Application Delivery Pipeline

```
Developer | GitHub | GitHub Actions / Jenkins | Gitleaks (secret scanning) |
SonarQube / Semgrep (SAST) | Dependency Scan (SCA) | Docker Build | Trivy Scan
(container scanning) | ECR | Argo CD (GitOps sync) | AWS EKS | Helm | Kubernetes |
Prometheus | Grafana | CloudWatch
```

18.2.2 Infrastructure Layer

```
Terraform / OpenTofu | AWS | VPC (Public + Private Subnets) | NAT Gateway - ALB - EKS
- ECR - RDS - S3 - IAM - CloudWatch
```

18.2.3 Security Layer Woven Throughout

- **Identity:** IAM (least privilege, roles over keys), KMS (encryption), Secrets Manager (credential storage/rotation)
- **Code & Build:** Trivy, SonarQube, Gitleaks, Checkov (IaC scanning)
- **Runtime App Testing:** OWASP ZAP (DAST)
- **Kubernetes:** RBAC, NetworkPolicy, Falco (runtime detection)
- **Cloud-wide Detection:** GuardDuty, Security Hub

18.3 Module 52: Interview Notes

Note

For every technology covered in this document, prepare answers in these four buckets: **(1) Concept Questions, (2) Scenario Questions, (3) Commands, (4) Real-Time Architecture Questions.** Representative examples across the whole stack are collected below.

18.3.1 1. Concept Questions

- What is Docker? How is it different from a Virtual Machine?
- What is Kubernetes, and why do we need an orchestrator at all?
- What is Terraform state, and why is remote state with locking important?
- What is CI/CD, and how does DevSecOps extend it?
- What is the difference between SAST, DAST, and SCA?
- What is the Shared Responsibility Model in AWS?
- What is RBAC, and how does it differ in cloud IAM vs Kubernetes?

- What is a Service Mesh, and when do you actually need one?
- What is an SBOM and why does supply chain security depend on it?
- What is the difference between an SLA, SLO, and SLI?

18.3.2 2. Scenario Questions

- A Pod is not starting – walk through your troubleshooting steps.
- Production CPU usage suddenly hits 100% – what do you check first?
- An AWS access key was accidentally leaked on GitHub – what is your immediate response?
- The Terraform state got deleted/corrupted – how do you recover?
- A deployment just went out and error rates spiked – how do you decide between rollback and fixing forward?
- A container keeps restarting (CrashLoopBackOff) right after a new image push – how do you isolate whether it's the image, the config, or the cluster?
- Your CI pipeline suddenly fails only on the security-scan stage – how do you triage whether it's a real vulnerability or a false positive?

18.3.3 3. Commands Cheat-Sheet

```
# kubectl
kubectl get pods -A
kubectl describe pod <pod>
kubectl logs <pod> -c <container> --previous
kubectl rollout undo deployment/<name>

# docker
docker build -t app:tag .
docker run -d -p 8080:80 app:tag
docker logs -f <container>

# terraform
terraform plan -out=tfplan
terraform apply tfplan
terraform state list

# aws cli
aws sts get-caller-identity
aws s3 ls
aws iam list-attached-user-policies --user-name <user>

# git
git log --oneline --graph
git diff HEAD~1
git cherry-pick <commit>

# helm
helm upgrade --install app ./chart -f values-prod.yaml

# linux
journalctl -u <service> -f
ss -tulpn
```

```
df -h && du -sh /var/log/*
```

18.3.4 4. Real-Time Architecture Questions

- How would you design a secure CI/CD pipeline from scratch, end to end?
- How would you deploy a highly available application on AWS across multiple AZs?
- How would you secure an EKS cluster for a regulated (e.g. PCI/HIPAA-adjacent) workload?
- How would you design secrets management across a multi-environment, multi-team pipeline?
- How would you design a disaster recovery strategy for a system with a 15-minute RPO and 1-hour RTO?
- How would you structure Terraform modules and state for a multi-account, multi-environment AWS organization?